

EECS 498 · AASE · FALL 2026

How Aider works

LAB01 · DESIGN YOUR PAIR-PROGRAMMER · SEP 14–15, 2026

Schedule

1. Aider deep dive
2. Hackathon 1
3. The next assignment
4. Office hours

The harness

Aider is a *harness*: a program that stands between you and a model.

The harness	The model
Picks the text the model sees	Reads it
Decides what a reply may do	Writes a reply
Edits files, then commits	Nothing

Take one apart now, build one later.

What the model never touches

- Your disk
- Your shell
- Your last session

Text in, text out. The harness does the rest.

What writes, and what does not

Arranges context

`/add`

`/read-only`

`/drop`

`/tokens`

`/ask`

`/diff`

Changes your files

`/code`

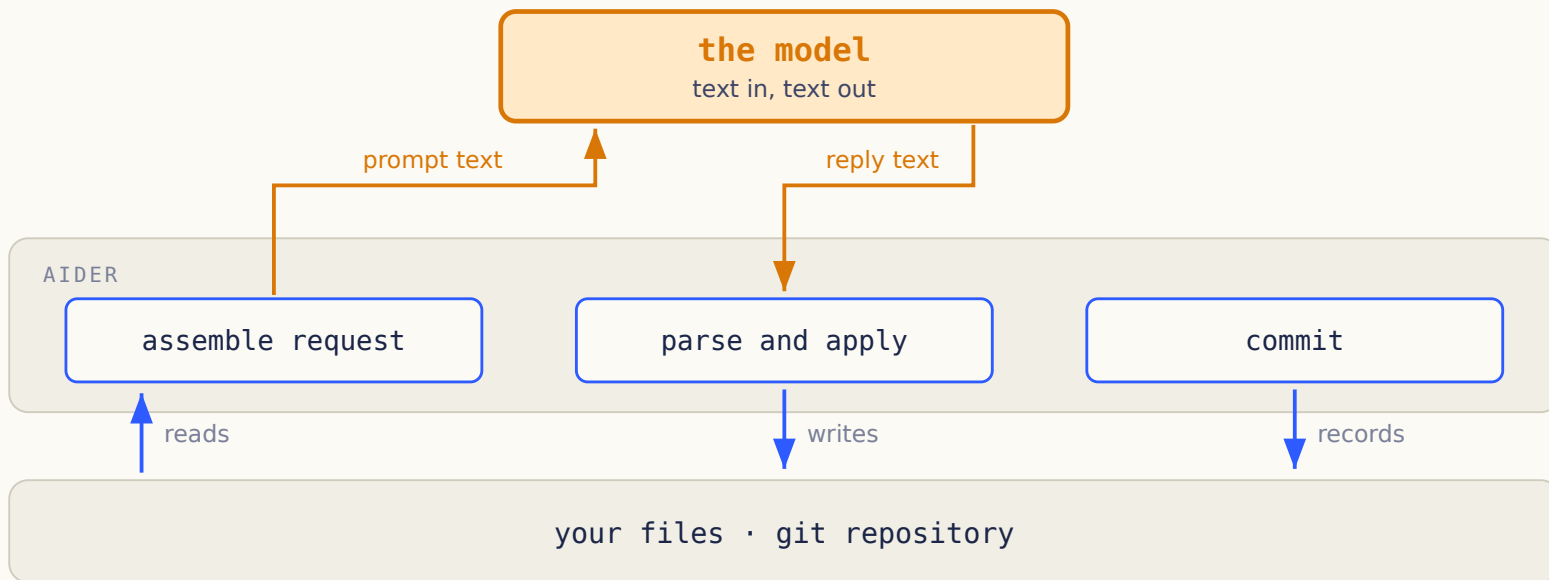
`/undo`

Bare text means `/code`. The default writes.

One turn

SELECTED FILES TO APPLIED EDIT

Aider, the model, and the repo



The model touches no file.

What goes into the request

Fixed each turn	Instructions, added files, repo map
Grows each turn	Session history, your message

Context is a selection. A file you never added is not in it.

File selection with /add

```
/add <file> ...  
/read-only <file> ...  
/tokens
```

Added files can be rewritten. Read-only files can only be cited.

Only these commands load a file.

The repo map

```
package/store.py:
:
|class Store:
|    def add(self, title, tags=()) -> Task:
|    def remove(self, task_id: int) -> bool:
:
|    def search(self, text, tag=None) -> list[Task]:
:

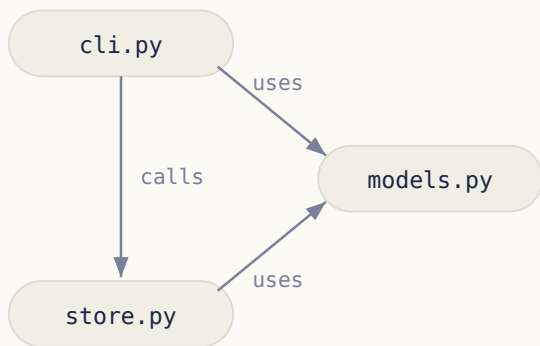
package/models.py:
|@dataclass
|class Task:
:
```

Signatures only. `:` is what was cut.

Files you never added.

Where the repo map comes from

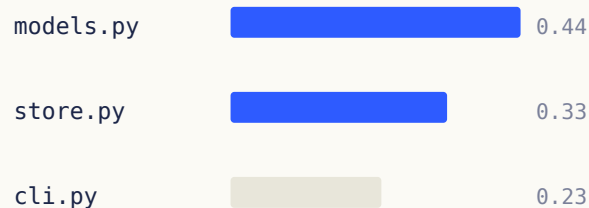
1. parse every file for what it defines and what it uses



models.py is used by both, so it wins

PageRank, rendered until the budget ends.

2. rank by what points at them



budget ends here

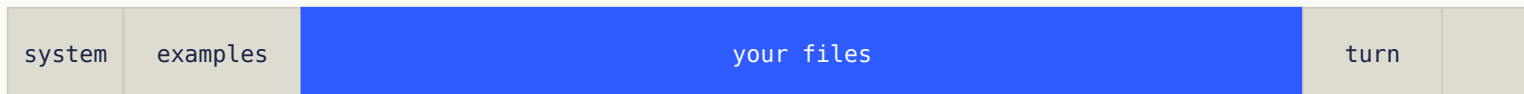
cli.py is left out entirely

What the map costs

repo map on



repo map off



0

the whole context window

Same window either way.

On a small model the map can outweigh the files.

One shared token budget



One budget: history, map, and files.

Edit-shaped replies

```
path/to/file.py
<<<<<<< SEARCH
the lines exactly as they appear on disk
=====
the lines that replace them
>>>>>>> REPLACE
```

One block, one file. Nothing forces the set to be complete.

The parse step

Parsing asks: is this a block?

A missing divider is a syntax error.

Validation asks: does it apply?

Unknown path, or zero exact matches.

Two questions. Two error messages.

Exact-match application

On disk

```
total = price * quantity
```

In the SEARCH block

```
total = price  * quantity
```

Two extra spaces. Zero matches.

Your contract: exact matches only, all blocks or none.

THURSDAY SEPTEMBER 24

Hackathon 1

- Two hours, your own project
- Prompt revealed in the room
- Submit before you leave
- Graded separately from the build

Bring a runnable increment.

Build due **Friday September 25, 11:59 PM.**

The pair-programmer

WHAT STAGE 1 ASKS FOR

A harness from scratch

Everything in the last half hour, written by you.

It holds	A set of files you chose
It asks	A model endpoint you configure
It writes	Only after you approve

Aider without Aider. Same job, your code.

The parts of the build

Part	What it does
Endpoint client	Talks to the model
Context policy	Decides what gets sent
Edit parser	Reads the reply
Approval gate	Writes, then commits

Seven features in the packet.

Where the requirements live

The packet in the project repository is the specification.

- `SPEC.md` for what it must do
- `RUBRIC.md` for how it is scored
- `CHECKLIST.md` for what you hand in

Due Friday September 25, 11:59 PM.

Architecture ownership

We specify

Observable behavior and safety rules

The API and configuration contract

Acceptance scenarios and the rubric

Where the course is going

You design

Responsibilities and who owns state

Internal interfaces and error flow

Tests, fixtures, increment boundaries

Where later capabilities enter

Different architectures can earn full marks.

Two sizes of spec

System spec, written once

Behavior, architecture, interfaces, failures.

Increment spec, one per change

Scoped context, ordered tasks, a checkable result.

Commit the design before the code.

The three diagrams

Diagram	Answers
Components and data flow	Who owns state?
Request sequence	Who does what in one edit?
Edit lifecycle	When can an edit be undone?

Mermaid is enough. Commit the first versions.

Rubric: the three buckets

Group	Points
Specification and diagrams	40
Behavior and verification	40
Evidence and reconciliation	20

`RUBRIC.md` carries the split, and grades you.

Model rules for specifications

- Drafting, critique, diagrams, code, tests
- 9B recommended, 4B permitted
- Secondary models recorded too
- Any compatible host allowed

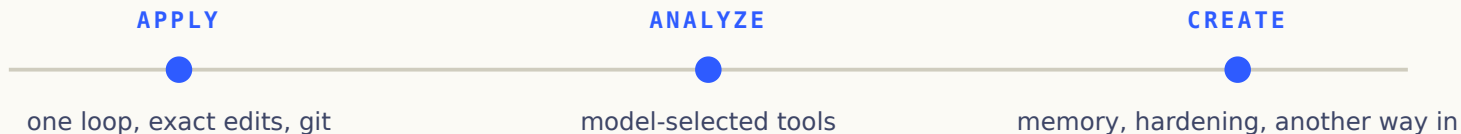
A different provider is not a different model.

What comes next

ONE REPOSITORY, GROWN UNTIL DECEMBER

THROUGH DECEMBER

One repository to December



Nothing is thrown away. Each phase grows the last.

WHERE THIS IS GOING

What an agentic CLI adds

Aider	An agentic CLI
You frame a bounded task	You hand over a broad one
You select the files	It discovers them
A fixed sequence of steps	The model picks the next tool
You check between tasks	Permissions bound it

BY DECEMBER

The harness by December

You will have built the kind of tool this lab took apart: a coding agent in the shape of Claude Code or OpenClaw, running against a model you configure, grown from the pair-programmer you start on Monday.

Today you drive one. In December you will have written one.

Office hours

THE REST OF THE SESSION IS YOURS

Questions?